Rashtreeya Sikshana Samithi Trust

R. V. COLLEGE OF ENGINEERING

[Autonomous Institution Affiliated to VTU, Belagavi]

Department of Computer Science & Engineering Bengaluru-560059

IOT & Embedded Computing – EL Report

Subject Code: CS344AI

IV SEMESTER B.E.

CSE

EL Report

[Autonomous Scheme 2022]

2025-2026

TRISHIELD

(A MULTI-LAYER BIOMETRIC SMART SAFE WITH DURESS AUTHENTICATION AND EMERGENCY ALERT SYSTEM)

PARINITHA M	**1RV24CS183**
P SAI LEKHYA	**1RV24CS181**
PARINITHA B S	**1RV24CS182**
NIKITHA DAYANAND	**1RV25CS415**

Rubrics

Criteria	Excellent	Good	Poor
Explanation of all Connections, Components & Demonstration 5 marks CO3	Clearly Demonstrates all the connections, Identifies all the components with I/Os **(5M)**	**Partially** Demonstrates all the connections, Identifies all the components with I/Os **(2-4M)**	**No understanding of** connections, Identifies wrongly the components **(0-1M)**
Demonstration of Cloud and Web/Mobile Interface 05 marks CO4, CO5	Fully implemented the web app/mobile app with full working demonstration **(5M)**	Partial implemented the web app/mobile app with full working demonstration **(2-4M)**	Not implemented the web app/mobile app **(0-1M)**
Documentation & Presentation: Project Report as per IoT Design Ten Steps 10 marks CO1,CO2	Well-structured report with clear explanations, figures, and references. **(10 M)**	Report is mostly clear but lacks some details/design steps **(5-8 M)**	Poorly structured report with missing design steps and wrong representation of steps. **(0-3 M)**

CIE – Laboratory (Part B) – IoT Based Project/Prototype Design & Development

Design & Demonstration of the Solution / Prototype (20 marks):

Marks Awarded: / 20Marks

Faculty Signature: **Signature of HOD**

Faculty Name: Neethu S

Date:

1. INTRODUCTION

With the rapid advancement of the Internet of Things (IoT), cloud computing, and biometric authentication technologies, traditional security systems are being replaced by intelligent access control solutions capable of providing higher levels of security, reliability, and real-time monitoring. Conventional locker systems primarily depend on mechanical keys, passwords, RFID cards, or a single biometric authentication method. Although these systems are easy to deploy, they remain vulnerable to key duplication, password theft, spoofing attacks, credential leakage, and unauthorized access. Moreover, most existing locker systems provide little or no cloud connectivity, centralized user management, or real-time monitoring, making them unsuitable for applications requiring enhanced security.

The **TriShield Smart Locker System** is designed as an IoT-enabled multi-factor authentication system that combines embedded systems, cloud computing, artificial intelligence, and biometric security into a unified access control platform. Instead of relying on a single authentication mechanism, the proposed system verifies the user's identity through multiple independent security layers consisting of **PIN verification**, **SMS-based One-Time Password (OTP) authentication**, **AI-powered facial recognition**, and **fingerprint verification**. Access to the locker is granted only after the successful completion of every authentication stage, significantly reducing the possibility of unauthorized access.

The proposed system adopts a **distributed embedded architecture**, where each hardware component performs a dedicated function. The **ESP32** serves as the master controller responsible for user interaction through the matrix keypad and LCD display, Wi-Fi communication, OTP generation, Twilio cloud messaging, and coordination of the authentication process. The **Arduino UNO** functions as the secondary controller, managing fingerprint authentication, additional servo-controlled locking mechanisms, and hardware-level security operations. Communication between the two controllers is established using the **I2C protocol**, enabling synchronized operation while distributing computational tasks efficiently.

Unlike conventional systems that store passwords directly within embedded devices, the proposed system utilizes **Firebase Realtime Database** as a secure cloud platform for centralized user management. User credentials are protected using the **SHA-256 cryptographic hashing algorithm**, ensuring that PIN values are never stored in plaintext. During authentication, the ESP32 communicates with a Python-based verification server over Wi-Fi, which securely validates the hashed PIN stored in Firebase before permitting further authentication. This cloud-based approach improves scalability, simplifies user management, and significantly enhances overall security.

To strengthen biometric verification, the system integrates **AI-based facial recognition** using a standard webcam, Python, OpenCV, and the Face Recognition library. During user registration, facial features are extracted as 128-dimensional face encodings and securely stored within Firebase. During authentication, live facial data captured through the webcam is compared against the stored facial encoding, allowing only registered users to proceed. This approach provides reliable identity verification without requiring specialized biometric camera hardware.

After successful PIN, OTP, and facial verification, the authentication process proceeds to **fingerprint verification** using the R307 fingerprint sensor connected to the Arduino UNO. The system supports both **normal fingerprint authentication** and a dedicated **duress fingerprint mechanism**. While the normal fingerprint grants complete access to the locker, the duress fingerprint appears to unlock the system under threat while simultaneously generating a silent emergency event and recording the incident in the cloud without alerting the intruder. This additional security feature improves user safety in high-risk situations.

The project also incorporates **real-time cloud logging** of every authentication attempt. Successful logins, failed authentications, fingerprint verification results, duress events, and system activities are automatically recorded in Firebase, enabling centralized monitoring and future forensic analysis. SMS-based OTP delivery is performed through the **Twilio Cloud API**, eliminating the need for dedicated GSM hardware while providing faster and more reliable communication over Wi-Fi.

Overall, the TriShield Smart Locker System demonstrates how embedded systems, IoT communication, cloud computing, computer vision, cryptographic security, and biometric authentication can be integrated into a single intelligent security platform. The developed prototype provides a scalable, reliable, and secure locker solution suitable for banks, educational institutions, research laboratories, hospitals, offices, industrial storage systems, and smart home applications where advanced multi-layer security and cloud-based monitoring are essential.

2. PROBLEM DEFINITION

2.1 Problem Statement

The increasing use of digital lockers in educational institutions, offices, laboratories, banks, and smart homes has created a growing demand for secure, intelligent, and remotely manageable access control systems. Conventional locker systems generally depend on mechanical keys, passwords, RFID cards, or a single biometric authentication mechanism such as fingerprint recognition. Although these methods provide basic security, they suffer from several limitations, including lost or duplicated keys, weak passwords, RFID cloning, biometric spoofing, and the absence of centralized monitoring.

Many existing IoT-enabled smart locker systems have improved remote accessibility by integrating internet connectivity and cloud platforms. However, most of these systems still rely on only one or two authentication methods, making them vulnerable if a single authentication factor is compromised. Furthermore, several existing solutions lack secure cloud-based credential management, encrypted storage of user information, real-time event logging, and intelligent biometric verification. In high-security environments, such vulnerabilities can lead to unauthorized access, theft of valuable assets, and poor traceability of security incidents.

Another major limitation of conventional systems is their inability to provide coordinated communication between embedded hardware, cloud services, and intelligent authentication software. Most systems either perform authentication locally without cloud synchronization or rely solely on cloud authentication without integrating hardware-level security. Additionally, OTP delivery through dedicated GSM modules increases hardware cost, maintenance, and power consumption.

To address these challenges, the proposed **TriShield Smart Locker System** introduces a cloud-enabled, IoT-based multi-factor authentication framework. The system verifies user identity through four independent authentication stages:

- Master PIN verification using a 4×3 matrix keypad
- SMS-based OTP verification using the Twilio Cloud API
- AI-based facial recognition using a webcam
- Fingerprint authentication using the R307 fingerprint sensor

The authentication process is coordinated through a distributed architecture consisting of an ESP32, an Arduino UNO, a Python-based face recognition server, and Firebase Realtime Database. User credentials are securely managed using SHA-256 hashing, while Firebase maintains centralized user records and authentication logs. This layered security approach significantly enhances protection against unauthorized access while enabling real-time monitoring, cloud synchronization, and future scalability.

2.2 Existing Limitations Identified

Advancements in the Internet of Things (IoT), embedded systems, cloud computing, and artificial intelligence have transformed conventional access control systems into intelligent security platforms capable of remote monitoring and automated authentication. Over the past decade, researchers have proposed several smart locker systems using technologies such as RFID, fingerprint sensors, OTP verification, cloud databases, and computer vision. These systems aim to improve security and user convenience while reducing dependence on traditional mechanical locks.

Early smart locker systems primarily relied on RFID cards or password-based authentication. Although these approaches simplified user access, they remained vulnerable to card duplication, password leakage, and unauthorized access. To overcome these limitations, researchers introduced biometric authentication using fingerprint sensors, providing stronger identity verification due to the uniqueness of human fingerprints. However, fingerprint-only systems still represent a single point of failure and are susceptible to spoofing attacks if additional security mechanisms are not implemented.

Recent IoT-based locker systems have integrated cloud databases such as Firebase to enable centralized user management, remote monitoring, and real-time synchronization. Firebase Realtime Database allows secure storage of user profiles, authentication logs, and device status while supporting communication between embedded devices and cloud applications. Nevertheless, many existing cloud-enabled systems continue to depend on only password or fingerprint authentication without implementing multiple independent security layers.

Computer vision has further expanded the capabilities of modern authentication systems through facial recognition. Using OpenCV and deep learning-based face recognition algorithms, webcams can accurately identify authorized users by comparing live facial feature encodings with previously registered data. Unlike dedicated biometric cameras, webcam-based recognition offers a low-cost and scalable solution suitable for educational and industrial applications.

Cloud communication platforms such as the Twilio REST API have also replaced traditional GSM modules for OTP delivery. Internet-based SMS services eliminate the need for dedicated cellular hardware, reduce implementation cost, and improve communication reliability while enabling secure OTP generation and delivery over Wi-Fi.

Despite these technological advancements, a review of recent literature indicates that most existing smart locker systems still provide limited authentication, minimal cloud integration, insufficient event logging, and poor coordination between embedded hardware and cloud services. Very few systems integrate PIN authentication, cloud-based OTP verification, AI-powered facial recognition, fingerprint authentication, cryptographic credential storage, and distributed embedded processing into a unified security framework.

The proposed **TriShield Smart Locker System** addresses these research gaps by combining embedded systems, IoT communication, cloud computing, cryptographic security, computer vision, and biometric authentication into a single intelligent access control platform. The integration of ESP32, Arduino UNO, Firebase Realtime Database, Twilio Cloud API, Python-based facial recognition, SHA-256 hashing, and fingerprint authentication provides a highly secure, scalable, and practical solution suitable for modern smart security applications.

3. OBJECTIVES

3.1. Primary Objectives

The primary objective of the **TriShield Smart Locker System** is to develop a secure, intelligent, and IoT-enabled locker system capable of protecting valuable assets through multiple independent authentication mechanisms while maintaining cloud connectivity, real-time monitoring, and reliable hardware-software integration. The project focuses on improving the security limitations of conventional locker systems by combining embedded systems, cloud computing, artificial intelligence, and biometric authentication into a single platform.

The primary objectives of the project are as follows:

3.1.1 To Develop a Multi-Factor Authentication System

The foremost objective of this project is to design and implement a robust multi-factor authentication framework that verifies the identity of the user through multiple independent security layers. Instead of relying on a single authentication method, the system performs sequential verification using:

- Master PIN entered through a 4×3 matrix keypad
- SMS-based One-Time Password (OTP) generated through Twilio Cloud API
- AI-powered facial recognition using a webcam

- Fingerprint verification using the R307 fingerprint sensor

This layered authentication significantly minimizes the possibility of unauthorized access while increasing the overall reliability of the locker system.

3.1.2 To Develop an IoT-Based Smart Locker System

Another important objective is to transform a conventional locker into an IoT-enabled intelligent security system. The ESP32 microcontroller provides Wi-Fi connectivity that allows the locker to communicate with cloud services, synchronize user data, and record authentication events in real time.

IoT connectivity enables:

- Remote cloud synchronization
- Centralized user management
- Real-time authentication logging
- Scalable system expansion
- Internet-based OTP communication

3.1.3 To Implement Secure Cloud-Based Authentication

The project aims to integrate Firebase Realtime Database as the cloud backend for storing and managing user information securely.

Firebase is used to maintain:

- Registered user information
- SHA-256 hashed PIN values
- Facial feature encodings
- Fingerprint ID mappings
- Authentication history
- Access logs
- Emergency events

Cloud integration ensures secure storage while eliminating the limitations of local memory-based authentication.

3.1.4 To Integrate AI-Based Facial Recognition

A major objective of the project is to develop an intelligent facial recognition module using Python, OpenCV, and the Face Recognition library.

The webcam captures the user's facial image during authentication, extracts facial feature encodings, and compares them with the encrypted facial data stored in Firebase. Only successfully recognized users are permitted to continue to the fingerprint authentication stage.

This additional biometric layer significantly strengthens overall system security.

3.1.5 To Develop a Distributed Embedded Architecture

Instead of using a single microcontroller for the entire system, the project adopts a distributed embedded architecture.

The responsibilities are divided as follows:

ESP32

- User interaction through keypad and LCD
- Wi-Fi communication
- Twilio SMS generation
- Firebase communication
- Servo-1 control
- Authentication coordination

Arduino UNO

- Fingerprint authentication
- Servo motor control
- Hardware-level security
- I2C communication with ESP32

This architecture improves system reliability, modularity, and processing efficiency.

3.1.6 To Automate the Locker Access Mechanism

The project aims to eliminate manual locking mechanisms by automating the entire locker operation using servo motors.

After successful completion of every authentication stage, the servo motors automatically unlock the locker compartments. If authentication fails at any stage, the locker remains securely locked.

Automation improves both user convenience and operational efficiency.

3.1.7 To Improve Security Through Cryptographic Techniques

The project implements SHA-256 hashing for secure PIN storage. Instead of storing plaintext passwords, the PIN is converted into a fixed-length cryptographic hash before being stored in Firebase.

During authentication, the entered PIN is hashed again and compared with the stored hash, ensuring that sensitive user credentials remain protected against unauthorized access.

3.2. Secondary Objectives

Apart from the primary objectives, the project also focuses on achieving several secondary goals that improve system performance, enhance learning outcomes, and increase the future scalability of the prototype.

3.2.1 To Improve User Convenience

The authentication process is designed to be simple and user-friendly through the use of an LCD interface, matrix keypad, automated OTP verification, facial recognition, and fingerprint authentication.

The entire authentication sequence requires minimal user interaction while maintaining high security.

3.2.2 To Gain Practical Knowledge of Embedded Systems

The project provides hands-on experience in embedded system design by integrating multiple hardware components, including ESP32, Arduino UNO, fingerprint sensors, servo motors, matrix keypad, LCD display, and communication protocols.

Students gain practical understanding of:

- Embedded programming
- Peripheral interfacing
- Hardware integration

- Real-time control systems

3.2.3 To Explore IoT and Cloud Computing

The project enables practical implementation of IoT concepts through cloud communication using Firebase and Twilio services.

Students understand:

- Cloud databases
- Real-time synchronization
- REST APIs
- Wi-Fi communication
- Remote authentication
- Cloud security

3.2.4 To Enhance Knowledge in Artificial Intelligence

By implementing facial recognition using Python and OpenCV, the project introduces practical concepts of computer vision and AI-based biometric authentication.

Students gain experience in:

- Image processing
- Facial feature extraction
- Face encoding
- Face comparison
- AI-assisted authentication

3.2.5 To Improve Hardware–Software Integration Skills

The project requires seamless communication between embedded hardware, desktop applications, cloud databases, and internet services.

This integration improves understanding of:

- I2C communication
- Serial communication
- Hardware-software synchronization
- Cloud-device interaction

- Distributed embedded architectures

3.2.6 To Develop a Scalable Smart Security Platform

The system is designed using a modular architecture so that additional security features can be incorporated in future without redesigning the existing hardware.

Possible future enhancements include:

- Mobile application control
- Voice authentication
- QR-code based access
- AI-based intrusion detection
- Smart surveillance cameras
- Remote locker management
- Multiple user profiles
- Real-time mobile notifications

3.2.7 To Develop an Industry-Oriented Prototype

The final objective is to create a practical prototype suitable for real-world applications such as:

- Banking lockers
- Educational institutions
- Research laboratories
- Hospitals
- Corporate offices
- Industrial storage units
- Smart homes
- Defence and military applications

The developed prototype demonstrates how IoT, embedded systems, cloud computing, biometrics, and artificial intelligence can be combined to build an intelligent and secure access control system suitable for future commercialization.

4. METHODOLOGY

4.1 APPROACH

The **TriShield Smart Locker System** follows a distributed embedded architecture that integrates IoT, cloud computing, biometric authentication, and computer vision to provide a secure multi-factor authentication system. The authentication process is executed sequentially, where the user must successfully pass every verification stage before the locker is unlocked.

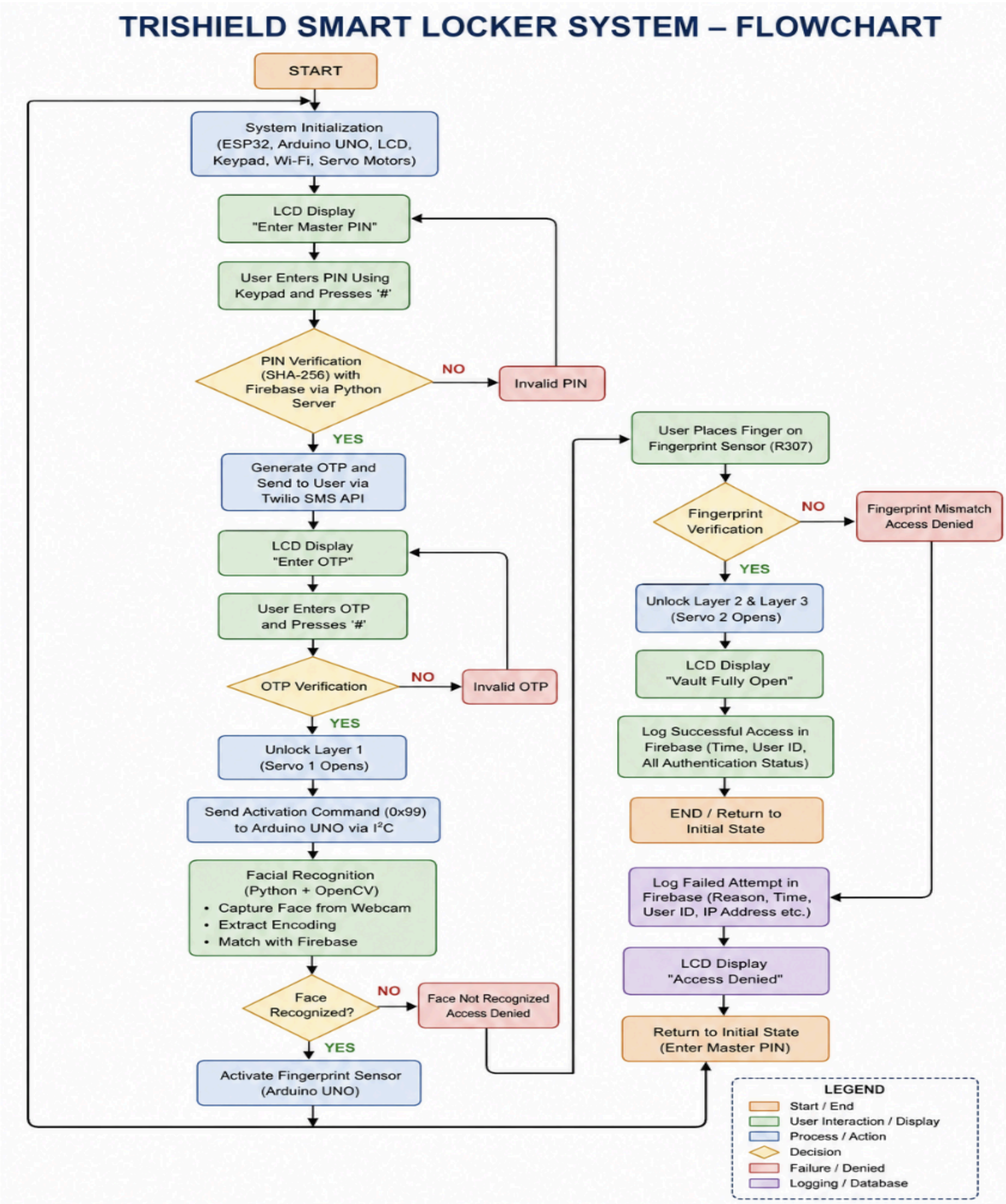
The system consists of three major components:

- **ESP32 (Master Controller):** Handles user interaction, PIN verification, Wi-Fi communication, Twilio SMS OTP generation, Firebase communication, LCD display, and controls the first-stage servo motor.
- **Arduino UNO (Security Controller):** Performs fingerprint authentication, controls the remaining servo motor(s), and communicates with the ESP32 through the I2C protocol.
- **Python Authentication Server:** Uses a webcam to perform facial recognition with OpenCV and the Face Recognition library. It communicates with Firebase to verify the user's facial encoding and records authentication events.

The complete authentication process follows these steps:

1. The user enters the **Master PIN** using the matrix keypad connected to the ESP32.
2. The ESP32 verifies the entered PIN using the stored SHA-256 hashed value.
3. If the PIN is correct, an OTP is generated and sent to the registered mobile number using the Twilio SMS API.
4. After successful OTP verification, the ESP32 unlocks the first servo and sends a trigger to the Arduino UNO through the I2C protocol.
5. The Python application activates the webcam and performs facial recognition by comparing the captured face with the facial encoding stored in Firebase.
6. If the face is verified successfully, the Arduino UNO activates the fingerprint sensor for biometric verification.
7. After successful fingerprint authentication, the remaining servo motor(s) unlock the locker, allowing access to the stored valuables.
8. Every authentication event is recorded in Firebase for future monitoring and analysis.

Flow Chart


TRISHIELD SMART LOCKER SYSTEM – FLOWCHART
START
System Initialization (ESP32, Arduino UNO, LCD, Keypad, Wi-Fi, Servo Motors)
LCD Display "Enter Master PIN"
User Enters PIN Using Keypad and Presses '#'
PIN Verification (SHA-256) with Firebase via Python Server
NO
Invalid PIN
YES
Generate OTP and Send to User via Twilio SMS API
LCD Display "Enter OTP"
User Enters OTP and Presses '#'
OTP Verification
NO
Invalid OTP
YES
Unlock Layer 1 (Servo 1 Opens)
Send Activation Command (0x99) to Arduino UNO via I²C
Facial Recognition (Python + OpenCV)
• Capture Face from Webcam
• Extract Encoding
• Match with Firebase
Face Recognized?
NO
Face Not Recognized Access Denied
YES
Activate Fingerprint Sensor (Arduino UNO)
User Places Finger on Fingerprint Sensor (R307)
Fingerprint Verification
NO
Fingerprint Mismatch Access Denied
YES
Unlock Layer 2 & Layer 3 (Servo 2 Opens)
LCD Display "Vault Fully Open"
Log Successful Access in Firebase (Time, User ID, All Authentication Status)
END / Return to Initial State
Log Failed Attempt in Firebase (Reason, Time, User ID, IP Address etc.)
LCD Display "Access Denied"
Return to Initial State (Enter Master PIN)
LEGEND
Start / End
User Interaction / Display
Process / Action
Decision
Failure / Denied
Logging / Database

4.2 Procedures

The development of the **TriShield Smart Locker System** was carried out through a structured methodology by dividing the project into multiple phases. Each phase focused on integrating hardware, software, cloud services, and security mechanisms to build a reliable multi-factor authentication system.

Phase 1: Requirement Analysis

The project began with identifying the limitations of conventional locker systems and studying existing IoT-based smart security solutions. Based on the literature survey, the requirements for the proposed system were finalized, including multi-factor authentication, cloud connectivity, biometric verification, and real-time monitoring. A distributed architecture using ESP32, Arduino UNO, and Python was selected to improve modularity and system performance.

Phase 2: Hardware Selection and Circuit Design

After defining the project requirements, suitable hardware components were selected based on functionality, cost, and compatibility. The ESP32 was chosen as the primary controller because of its built-in Wi-Fi capability, while the Arduino UNO was used to manage the fingerprint sensor and servo motor operations. A matrix keypad was incorporated for PIN entry, an LCD module was used for displaying system prompts, a webcam was selected for facial recognition, and an R307 fingerprint sensor was used for biometric verification. The complete circuit was designed with proper power distribution and I2C communication between the ESP32 and Arduino UNO.

Phase 3: Software Development

The software was developed in multiple modules to simplify debugging and future scalability.

The ESP32 firmware was programmed using the Arduino IDE to handle user interaction through the keypad and LCD, generate OTPs, communicate with Firebase, send SMS notifications using the Twilio API, and coordinate the authentication sequence.

The Arduino UNO firmware was developed to perform fingerprint enrollment, fingerprint verification, and servo motor control. Communication between the ESP32 and Arduino UNO was established using the I2C protocol.

A Python application was developed to perform facial recognition using OpenCV and the Face Recognition library. The application captures images from the webcam, extracts facial feature encodings, compares them with the registered user's data stored in Firebase, and returns the authentication result to the embedded system.

Phase 4: Cloud Integration

Firebase Realtime Database was configured as the central cloud platform for storing and managing user information. During user registration, the system stores the user's profile details, SHA-256 hashed PIN, facial encodings, fingerprint ID, and authentication history.

Twilio Cloud API was integrated with the ESP32 to generate and send One-Time Passwords (OTP) to the registered mobile number over Wi-Fi. This eliminates the need for a GSM module while providing faster and more reliable communication.

Phase 5: System Integration

Once all hardware and software modules were developed individually, they were integrated into a single authentication framework.

The ESP32 first verifies the user's PIN and OTP. After successful verification, it unlocks the first locking stage and sends an I2C command to the Arduino UNO. Simultaneously, the Python application activates the webcam for facial recognition. Once the face is verified, the Arduino UNO performs fingerprint authentication. If all authentication stages are successful, the remaining servo motor(s) unlock the locker, granting secure access.

This modular integration ensured smooth communication between the cloud, desktop application, and embedded controllers.

Phase 6: Testing and Validation

The complete system was tested under various operating conditions to evaluate its reliability and security. Individual hardware modules such as the keypad, LCD, fingerprint sensor, servo motor, webcam, and Wi-Fi communication were tested independently before integrating them into the complete system.

Different authentication scenarios were evaluated, including:

- Correct and incorrect PIN entry
- Valid and invalid OTP verification
- Authorized and unauthorized facial recognition
- Successful and failed fingerprint authentication

- Cloud synchronization with Firebase
- SMS delivery using Twilio
- Communication between ESP32 and Arduino UNO through I2C

The system successfully authenticated authorized users while preventing unauthorized access. Authentication events were recorded in Firebase, and the overall response time of the system was found to be reliable for real-time smart locker applications.

Project Timeline

Phase	Activity
Phase 1	Requirement Analysis and Literature Review
Phase 2	Hardware Selection and Circuit Design
Phase 3	Software Development (ESP32, Arduino UNO & Python)
Phase 4	Firebase and Twilio Cloud Integration
Phase 5	Hardware–Software Integration
Phase 6	Testing, Validation and Performance Evaluation

5. PROJECT EXECUTION

5.1 PLANNING AND DESIGN

The development of the **TriShield Smart Locker System** began with identifying the limitations of existing smart locker systems and proposing a more secure multi-factor authentication approach. Initially, several authentication methods and hardware configurations were studied to determine the most reliable and scalable solution.

A distributed architecture was selected to divide the system into independent modules. The **ESP32** was chosen as the master controller because of its built-in Wi-Fi capability and ability to communicate with cloud services. The **Arduino UNO** was used as the secondary controller to handle fingerprint authentication and servo motor operations. Firebase Realtime Database was selected for cloud storage, while the Twilio Cloud API was used for SMS-based OTP delivery. Python, OpenCV, and the Face Recognition library were chosen for implementing facial recognition using a standard webcam.

The hardware layout was carefully designed to ensure smooth communication between all components. I2C communication was established between the ESP32 and Arduino UNO for synchronization of authentication events. The overall system architecture was planned to be modular, allowing future expansion without major hardware modifications.

The software architecture was also divided into separate modules:

- **ESP32 Module:** PIN verification, OTP generation, LCD interface, Wi-Fi communication, Firebase integration, and Servo-1 control.
- **Arduino UNO Module:** Fingerprint enrollment, fingerprint verification, and servo motor control.
- **Python Module:** Facial recognition, Firebase communication, authentication management, and cloud logging.
- **Cloud Module:** User profile management, SHA-256 hashed PIN storage, facial encodings, fingerprint mapping, and access logs.

This modular design simplified debugging, testing, and maintenance while improving overall system reliability.

5.2 IMPLEMENTATION

After completing the planning phase, the hardware and software modules were implemented individually before integrating them into a complete authentication system.

The ESP32 firmware was developed using the Arduino IDE to control the matrix keypad, LCD display, Wi-Fi communication, Twilio SMS service, and Firebase connectivity. When the user enters the Master PIN through the keypad, the ESP32 verifies the entered PIN against the SHA-256 hashed value stored in Firebase. If the verification is successful, an OTP is generated and sent to the registered mobile number using the Twilio Cloud API. After the correct OTP is entered, the ESP32 unlocks the first-stage servo motor and sends an I2C trigger to the Arduino UNO.

The Arduino UNO firmware was developed to interface with the R307 fingerprint sensor and the remaining servo motor. Upon receiving the trigger from the ESP32, the Arduino UNO waits for the user to place their finger on the sensor. The captured fingerprint is compared with the enrolled templates stored within the fingerprint module. If a valid match is found, the corresponding servo motor unlocks the locker. Otherwise, access is denied.

The facial recognition module was implemented using Python with OpenCV and the Face Recognition library. A webcam continuously captures the user's image during authentication. The captured face is converted into facial feature encodings and compared with the registered face encodings stored in Firebase. If the face is successfully recognized, the system proceeds to fingerprint authentication; otherwise, the authentication process is terminated.

Firebase Realtime Database was integrated with the Python application to maintain centralized user information, hashed PIN values, facial encodings, fingerprint ID mappings, and authentication history. Twilio Cloud API was integrated to provide reliable SMS-based OTP delivery over Wi-Fi, eliminating the need for a GSM module.

Finally, all hardware modules, software components, and cloud services were integrated and tested together. Communication between ESP32, Arduino UNO, Python application, and Firebase was verified to ensure smooth synchronization. The final prototype successfully demonstrated secure multi-factor authentication with cloud connectivity and automated locker access.

6. TOOLS AND TECHNIQUES USED

6.1 TOOLS

- The development of the TriShield Smart Locker System involved various hardware components, software platforms, cloud services, and development tools.

Tool / Component	Purpose
ESP32	Acts as the master controller for PIN verification, Wi-Fi communication, OTP generation, Firebase integration, and Servo-1 control.
Arduino UNO	Controls fingerprint authentication and servo motor operation while communicating with the ESP32 through I2C.
Matrix Keypad (4×3)	Used for secure Master PIN entry during authentication.
16×2 LCD Display (I2C)	Displays user instructions, authentication status, and system messages.
R307 Fingerprint Sensor	Performs biometric fingerprint enrollment and verification.
Servo Motor(s)	Controls the locking and unlocking mechanism of the smart locker.
Webcam	Captures live images for facial recognition during authentication.

Firebase Realtime Database	Stores user information, SHA-256 hashed PINs, facial encodings, fingerprint mappings, and authentication logs.
Twilio Cloud API	Generates and sends SMS-based OTPs to registered mobile numbers.
Arduino IDE	Used for programming the ESP32 and Arduino UNO.
Visual Studio Code	Used for Python development and Firebase integration.
Python 3	Implements facial recognition and cloud communication.
OpenCV	Performs image acquisition and preprocessing for facial recognition.
Face Recognition Library	Generates and compares facial feature encodings.
PySerial	Enables serial communication between the Python application and embedded controllers.

6.2 TECHNIQUES

Several modern techniques were implemented to improve the security, reliability, and efficiency of the proposed smart locker system.

Multi-Factor Authentication (MFA)

The system uses four independent authentication layers: Master PIN verification, SMS-based OTP, facial recognition, and fingerprint authentication. This significantly reduces the risk of unauthorized access.

Internet of Things (IoT)

The ESP32 provides Wi-Fi connectivity, enabling communication with Firebase and Twilio Cloud services. This allows centralized user management, cloud synchronization, and remote authentication.

Cloud Computing

Firebase Realtime Database serves as the cloud backend for securely storing user profiles, authentication records, facial encodings, fingerprint mappings, and encrypted PIN values. Cloud storage also enables real-time synchronization and access logging.

Computer Vision

Facial recognition is implemented using OpenCV and the Face Recognition library. The webcam captures live facial images, extracts facial feature encodings, and compares them with the registered user's data stored in Firebase.

Biometric Authentication

Fingerprint verification is performed using the R307 fingerprint sensor. Only enrolled fingerprints are permitted to access the locker, providing a reliable biometric security layer.

Cryptographic Security

The system protects user PINs using the SHA-256 hashing algorithm before storing them in Firebase. During authentication, the entered PIN is hashed again and compared with the stored hash, preventing exposure of plaintext credentials.

I2C Communication

The ESP32 and Arduino UNO communicate using the I2C protocol, enabling synchronized operation between cloud-based authentication and hardware-level fingerprint verification while reducing wiring complexity.

Cloud-Based OTP Verification

Instead of using a GSM module, the system generates and delivers One-Time Passwords through the Twilio Cloud API over Wi-Fi. This approach improves communication reliability while reducing hardware cost and maintenance requirements.

These tools and techniques collectively enable the TriShield Smart Locker System to provide a secure, scalable, and intelligent access control solution suitable for modern IoT-based security applications.

7. Results

7.1 System Implementation Results

The proposed **TriShield Smart Locker System** was successfully designed, developed, and tested as a complete IoT-enabled multi-factor authentication system. The integration of embedded hardware, cloud services, and software applications resulted in a secure and reliable access control solution.

The ESP32 successfully managed the user interface by controlling the matrix keypad, LCD display, Wi-Fi communication, Twilio SMS service, Firebase connectivity, and the first-stage servo motor. The Arduino UNO communicated seamlessly with the ESP32 through the I2C protocol and successfully performed fingerprint authentication and servo motor control.

The Python application accurately implemented facial recognition using OpenCV and the Face Recognition library. Live facial images captured through the webcam were successfully matched with the registered facial encodings stored in Firebase. The complete authentication process operated sequentially without communication errors between the ESP32, Arduino UNO, Python application, and Firebase Realtime Database.

7.2 Experimental Results

The developed prototype was evaluated under multiple authentication scenarios to verify the functionality of each module as well as the overall system performance.

The system successfully performed:

- Master PIN verification using the matrix keypad.
- SHA-256 based PIN validation through Firebase.
- SMS-based OTP generation and delivery using the Twilio Cloud API.

- Facial recognition using a webcam and Python.
- Fingerprint verification using the R307 fingerprint sensor.
- Automatic locker unlocking using servo motors after successful authentication.
- Real-time synchronization of authentication logs with Firebase.
- Reliable communication between ESP32 and Arduino UNO using the I2C protocol.

Authentication was granted only when every verification stage was completed successfully. Any failure during PIN verification, OTP validation, face recognition, or fingerprint authentication immediately terminated the authentication process and prevented unauthorized access.

7.3 Performance Analysis

The overall performance of the proposed system was evaluated based on authentication accuracy, communication reliability, cloud synchronization, and hardware response.

Parameter	Observation
PIN Authentication	Successfully verified using SHA-256 encrypted credentials stored in Firebase
SMS OTP Verification	OTP generated and delivered successfully through Twilio Cloud API
Face Recognition	Registered users were accurately identified using webcam-based facial recognition
Fingerprint Authentication	Successful matching of enrolled fingerprints with high reliability
Cloud Synchronization	User credentials and authentication logs updated correctly in Firebase
ESP32–Arduino Communication	Stable I2C communication without data loss

Servo Motor Operation	Smooth locking and unlocking of the locker mechanism
Overall System Performance	Reliable and stable operation during repeated authentication cycles

7.4 Result Analysis

The results demonstrate that the proposed **TriShield Smart Locker System** successfully achieves secure multi-factor authentication by combining IoT, cloud computing, artificial intelligence, and biometric technologies.

The distributed architecture using ESP32 and Arduino UNO reduced the processing load on individual controllers while improving communication efficiency. Firebase Realtime Database provided centralized user management and secure cloud storage, whereas the Twilio API enabled reliable OTP delivery without requiring additional GSM hardware. The integration of facial recognition and fingerprint authentication significantly strengthened system security compared to traditional smart locker systems.

Overall, the developed prototype satisfied all project objectives by providing a secure, scalable, and intelligent locker system suitable for applications in banks, educational institutions, laboratories, offices, hospitals, and smart home environments.

8. Prototype (Hardware/Software)

8.1 Prototype Description

The **TriShield Smart Locker System** prototype is a fully functional IoT-enabled security system developed to provide secure access through multi-factor authentication. The prototype integrates embedded hardware, cloud computing, and artificial intelligence to create a reliable and intelligent locker system.

The system consists of an **ESP32** acting as the master controller, an **Arduino UNO** functioning as the secondary controller, a **4×3 matrix keypad**, **16×2 I2C LCD display**, **R307 fingerprint sensor**, **servo motor**, and a **webcam** for facial recognition. Firebase Realtime Database is used as the cloud backend to store user credentials, facial encodings, authentication history, and encrypted PIN values, while the **Twilio Cloud API** is used to deliver SMS-based OTPs during authentication.

The authentication process begins with Master PIN verification, followed by OTP verification, facial recognition, and fingerprint authentication. Only after successful completion of all authentication stages is the locker unlocked automatically through the servo motor. This layered security architecture makes the prototype suitable for applications requiring high levels of security.

8.2 Development Process

The prototype was developed using a modular approach, where each hardware and software component was implemented and tested individually before complete system integration.

Initially, the ESP32 was programmed to interface with the keypad, LCD display, Wi-Fi module, Firebase, and Twilio API. The Arduino UNO was then configured to communicate with the fingerprint sensor and servo motor while maintaining communication with the ESP32 through the I2C protocol.

The facial recognition module was developed separately using Python, OpenCV, and the Face Recognition library. Registered users' facial encodings were securely stored in Firebase and retrieved during authentication for comparison with live webcam images.

After validating the individual modules, all components were integrated into a unified authentication framework. Communication between the ESP32, Arduino UNO, Python application, and Firebase was optimized to ensure reliable synchronization and smooth execution of the authentication sequence. Multiple rounds of debugging and testing were performed to improve system stability and reduce communication delays.

8.3 Testing and Validation

The completed prototype was tested under various authentication scenarios to verify the correctness and reliability of the proposed system. Each authentication stage was evaluated independently before performing complete end-to-end testing.

The following test cases were successfully validated:

Test Scenario	Expected Outcome	Result

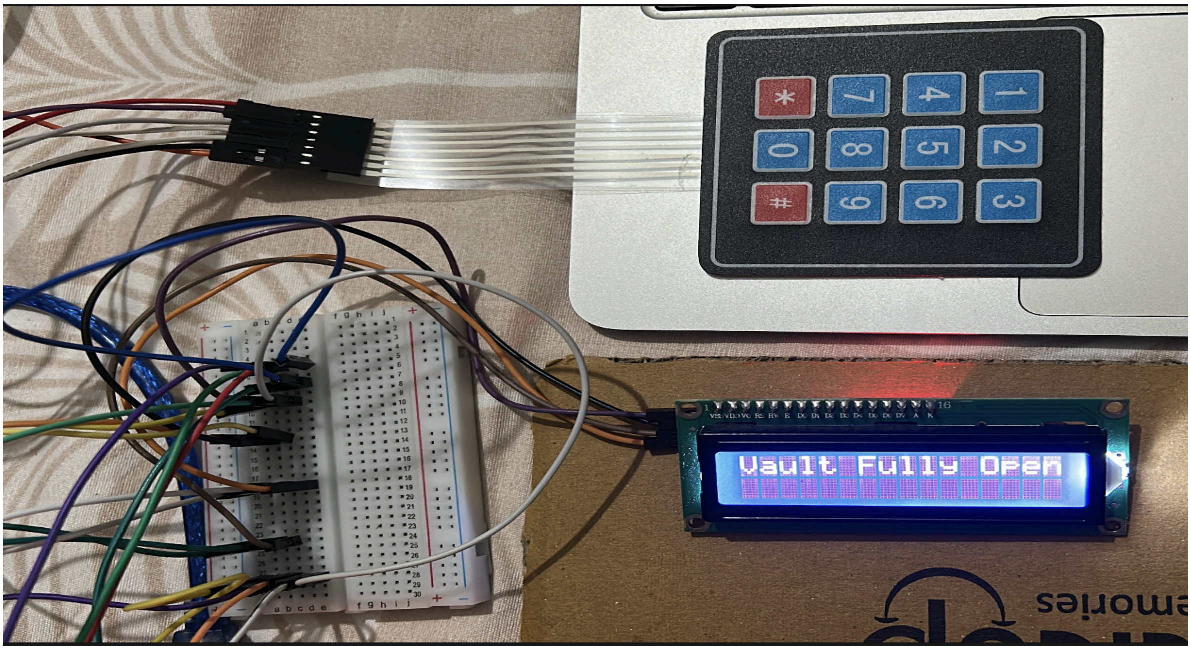
Correct Master PIN	Proceed to OTP verification	Passed
Incorrect Master PIN	Authentication terminated	Passed
Valid SMS OTP	Proceed to face recognition	Passed
Invalid SMS OTP	Access denied	Passed
Authorized Face	Proceed to fingerprint verification	Passed
Unauthorized Face	Authentication terminated	Passed
Registered Fingerprint	Locker unlocked	Passed
Unregistered Fingerprint	Access denied	Passed
Firebase Synchronization	Authentication data updated	Passed
I2C Communication	ESP32 and Arduino synchronized	Passed
Servo Motor Operation	Locker opened and closed correctly	Passed

The testing results confirmed that the proposed smart locker system performs reliably under normal operating conditions. All authentication modules worked together successfully, cloud synchronization was completed without errors, and the overall response time was suitable for real-time smart security applications.

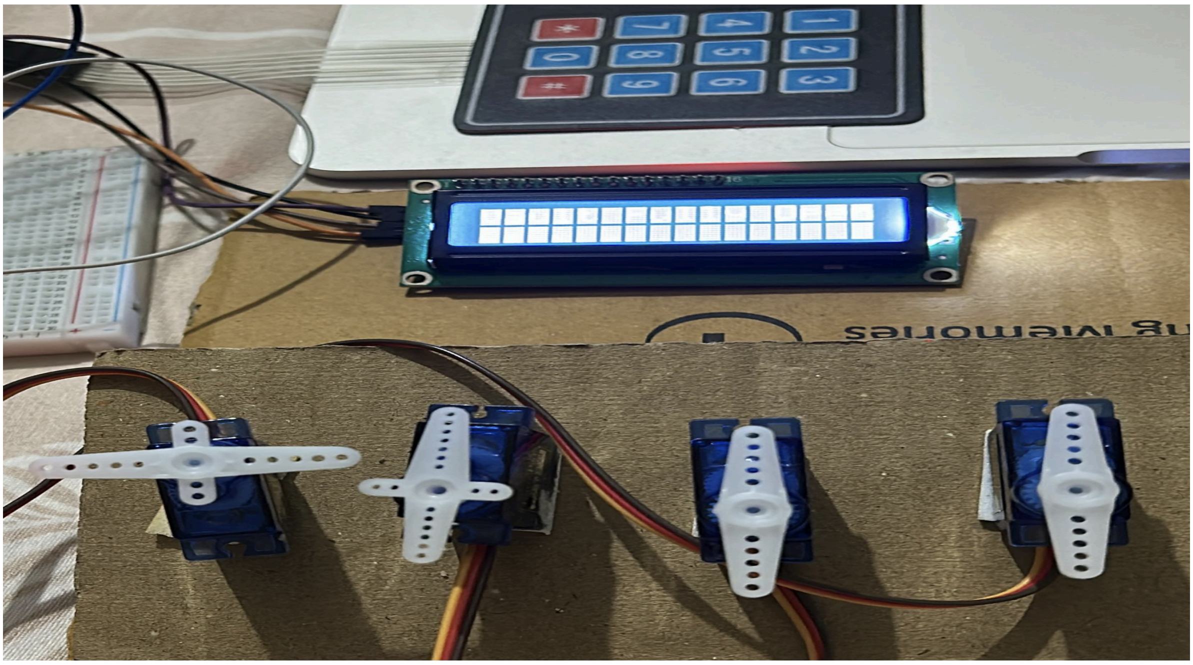
8.4 Prototype Features

The final prototype provides the following features:

1. Multi-factor authentication using PIN, OTP, facial recognition, and fingerprint verification.
2. IoT-enabled cloud communication through Firebase Realtime Database.
3. SMS-based OTP delivery using the Twilio Cloud API.
4. Real-time facial recognition using a webcam and OpenCV.
5. Secure storage of user credentials using SHA-256 hashing.
6. Reliable communication between ESP32 and Arduino UNO using the I2C protocol.
7. Automated locker operation using servo motor control.
8. Cloud-based authentication logging and user management.
9. Modular architecture allowing future expansion and integration of additional security features.



In case of all the correct entries and the normal finger

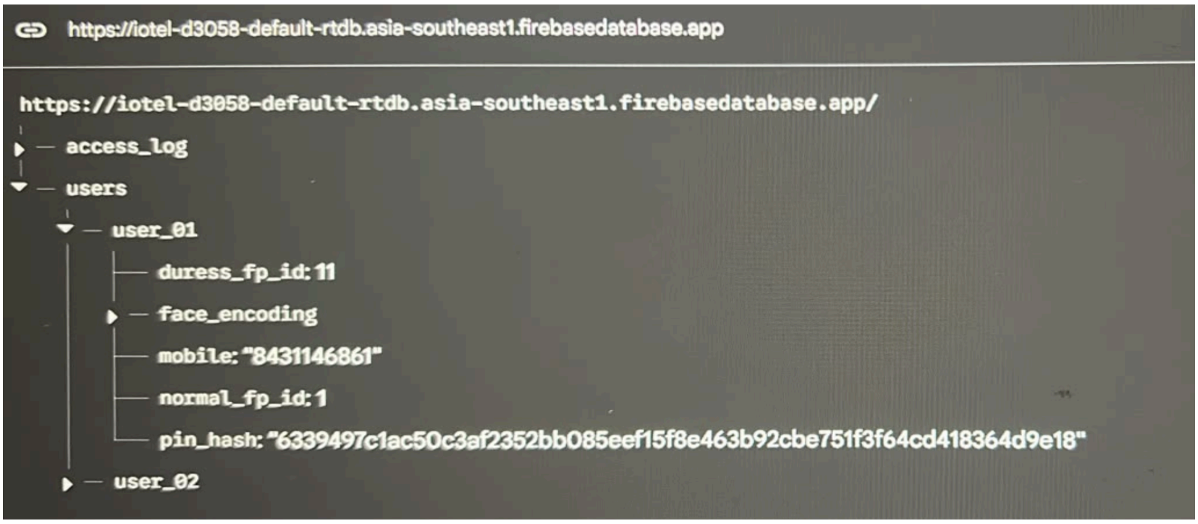


One servo motor opened after one level of authentication

Software Prototype

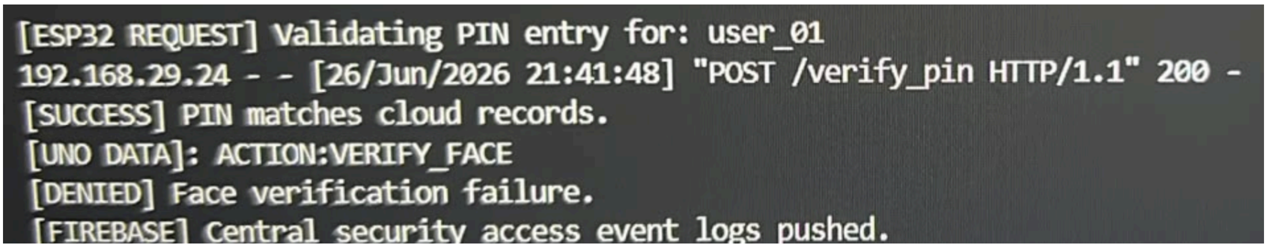
Include screenshots of:

- Firebase Database
- VS Code Project
- Python Face Recognition Running
- Twilio SMS Received
- Serial Monitor
- Face Registration Window
- Authentication Successful Screen



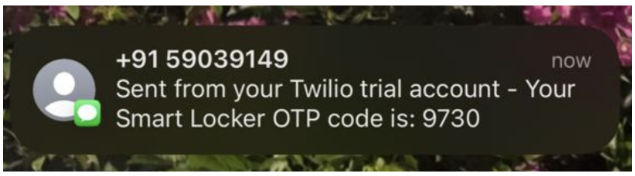
Firebase Realtime Database

Firebase stores all registered user information including hashed PIN values, facial encodings, fingerprint ID mappings, and authentication history. It acts as the central cloud database for the smart locker system.



Python Face Recognition Mode when the face recognition fails

The Python application captures live video through a webcam, detects the user's face using OpenCV, generates facial encodings, compares them with the stored Firebase database, and returns the authentication result.



Twilio OTP Message

After successful PIN verification, the ESP32 communicates with the Twilio Cloud API over Wi-Fi to generate and deliver a One-Time Password (OTP) to the registered user's mobile number.

```
[ESP32 REQUEST] Validating PIN entry for: user_01
192.168.29.24 - - [26/Jun/2026 21:40:33] "POST /verify_pin HTTP/1.1" 200 -
[SUCCESS] PIN matches cloud records.
[UNO DATA]: ACTION:VERIFY_FACE
[UNO DATA]: [SUCCESS] Layer 2 Unlocked.
[UNO DATA]: PROMPT: Scan fingerprint to open vault.
[UNO DATA]: [INFO] Authentication clear. Vault open.
[FIREBASE] Central security access event logs pushed.
```

Authentication Log when the complete process happens and all servos open

The authentication process is displayed through the serial monitor, showing successful PIN verification, OTP validation, facial recognition, fingerprint matching, and locker unlocking.

8.3 Important Code Snippets

A. Wi-Fi Connection (ESP32)

```
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {

  delay(500);

}

Serial.println("WiFi Connected");
```

Description

This code establishes Wi-Fi connectivity between the ESP32 and the local network, enabling communication with Firebase and Twilio cloud services.

B. OTP Generation

```
generatedOTP = String(random(1000,9999));

sendOtpSms(generatedOTP);
```

Description

A random four-digit OTP is generated and transmitted to the registered mobile number using the Twilio Cloud API.

C. Servo Control

```
servo1.write(90);

delay(2000);

servo1.write(0);
```

Description

The servo motor unlocks the locker after successful completion of the required authentication stages.

D. Fingerprint Verification

```
result = finger.fingerSearch();


if(result == FINGERPRINT_OK){

  Serial.println("Fingerprint Matched");

}
```

Description

The fingerprint captured by the R307 sensor is compared with the enrolled templates. Only registered fingerprints are allowed to proceed.

E. Face Recognition

```
face_locations = face_recognition.face_locations(frame)

encodings = face_recognition.face_encodings(frame, face_locations)
```

Description

The webcam captures a live image, detects the user's face, and extracts facial feature encodings for comparison with the registered user data.

F. Firebase Authentication

```
db.reference("Users").child(user_id).get()
```

Description

This code retrieves the registered user details from Firebase Realtime Database for authentication.

G. Twilio SMS

```
http.POST(httpRequestData);
```

Description

The ESP32 sends an HTTP POST request to the Twilio REST API to deliver the generated OTP via SMS.

H. I2C Communication

```
Wire.beginTransmission(UNO_I2C_ADDRESS);

Wire.write(0x99);

Wire.endTransmission();
```

Description

After OTP verification, the ESP32 sends an I2C command to the Arduino UNO, initiating the fingerprint authentication stage.

9. Conclusion

9.1 Summary

The **TriShield Smart Locker System** successfully demonstrates the implementation of a secure, intelligent, and IoT-enabled access control system by integrating embedded systems, cloud computing, biometric authentication, and artificial intelligence. The project addresses the limitations of conventional locker systems by introducing a multi-factor authentication framework consisting of **Master PIN verification, SMS-based OTP authentication, facial recognition, and fingerprint verification**.

The system was implemented using an **ESP32** as the master controller, an **Arduino UNO** as the security controller, a **matrix keypad**, **LCD display**, **R307 fingerprint sensor**, **servo motor**, and a **webcam** for facial recognition. Firebase Realtime Database was used for secure cloud storage of user credentials and authentication logs, while the Twilio Cloud API enabled reliable SMS-based OTP delivery.

The complete authentication process was successfully validated through experimental testing. Communication between the ESP32, Arduino UNO, Python application, and Firebase remained stable throughout the authentication sequence. The final prototype demonstrated reliable performance, accurate biometric verification, secure cloud synchronization, and automated locker control, thereby fulfilling the primary objectives of the project.

Overall, the TriShield Smart Locker System provides a practical and scalable solution for modern smart security applications where enhanced protection, cloud connectivity, and intelligent authentication are required.

9.2 Limitations

Although the developed system performs reliably, a few limitations were observed during implementation:

- The system requires a stable Wi-Fi connection for Firebase synchronization and Twilio SMS services.
- Facial recognition performance may be affected under poor lighting conditions or improper camera positioning.
- Authentication time increases slightly due to the sequential execution of multiple verification stages.
- The current prototype supports a limited number of registered users and can be further optimized for large-scale deployments.
- Cloud-based services introduce dependency on internet availability for OTP generation and data synchronization.

Despite these limitations, the system provides significantly higher security compared to conventional locker systems.

9.3 Future Scope

The proposed smart locker system can be further enhanced by incorporating additional security features and intelligent automation.

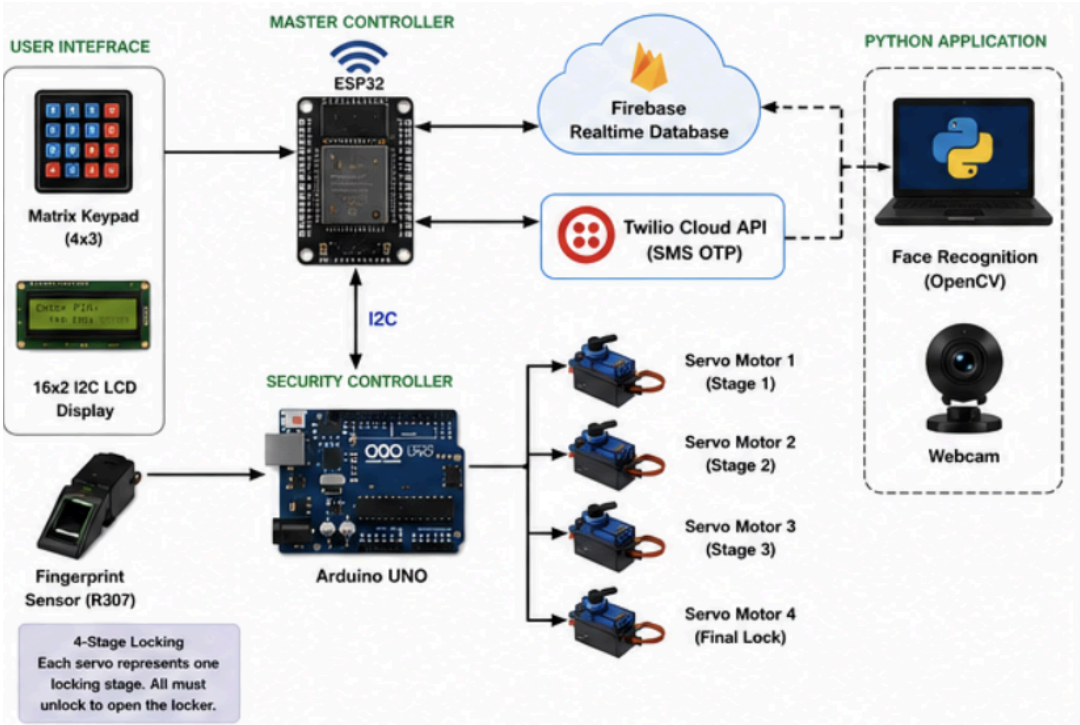
Possible future enhancements include:

- Development of an Android or iOS mobile application for remote locker monitoring and management.
- Integration of push notifications and email alerts for authentication events and security breaches.
- AI-based intrusion detection using machine learning algorithms.
- Voice recognition or iris recognition as additional authentication layers.
- Battery backup and power management for uninterrupted operation during power failures.
- Real-time surveillance using IP cameras integrated with the locker system.
- Cloud analytics dashboard for monitoring authentication history and system usage.
- Multi-user role management with different access privileges.
- Integration with smart home and industrial automation platforms.
- Commercial deployment using dedicated PCB design and compact embedded hardware.

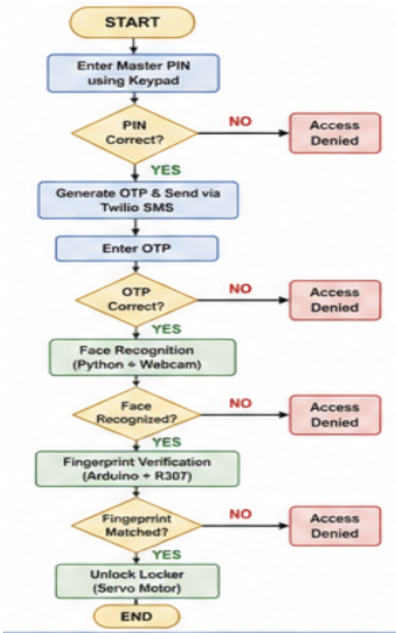
The modular architecture of the proposed system allows these improvements to be implemented without significant modifications to the existing hardware and software infrastructure.

10. VISUALS

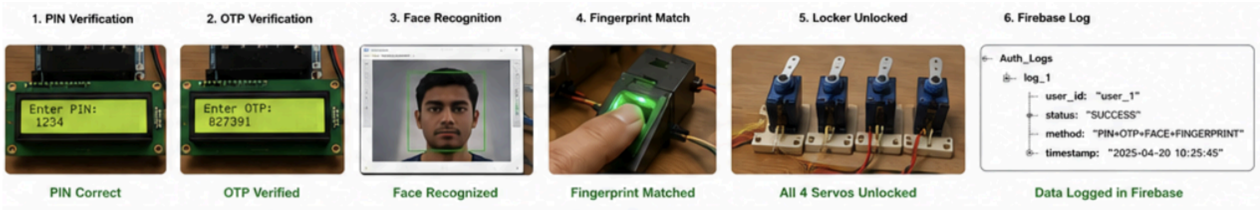
10.1 System architecture :



10.2 Authentication flowchart :



10.3 Authentication Results



11. Outcome of the Work

The TriShield Smart Locker System successfully demonstrates the practical implementation of an IoT-enabled, cloud-connected smart security solution that combines embedded systems, biometric authentication, and artificial intelligence into a single integrated platform.

Outcomes Achieved

- Designed and developed a functional multi-factor authentication smart locker prototype.
- Successfully implemented secure PIN verification using SHA-256 hashing.
- Integrated SMS-based OTP generation and verification using the Twilio Cloud API.
- Developed real-time facial recognition using Python, OpenCV, and a webcam.
- Implemented fingerprint authentication using the R307 fingerprint sensor.
- Established reliable communication between the ESP32 and Arduino UNO through the I2C protocol.
- Integrated Firebase Realtime Database for centralized user management and cloud-based authentication logging.
- Automated the locker locking and unlocking mechanism using servo motor control.
- Successfully tested and validated the complete authentication workflow under different operating conditions.

Applications

The developed prototype can be adapted for use in:

- Banks and financial institutions
- Educational institutions
- Research laboratories
- Hospitals and healthcare facilities
- Corporate offices
- Industrial storage units
- Smart homes
- Government and defence organizations

Scope for Product Development

The proposed system has strong potential for commercialization due to its modular architecture and cloud-based design. Future versions can include a dedicated mobile application, advanced AI-based security features, commercial PCB design, battery backup, and integration with smart building management systems, making it suitable for deployment as a next-generation intelligent locker solution.